Along with all the modifications I want to do with the BDHS 2022 paper,

The next step is cross-validation.

4️⃣Used 10-fold cross-validation with AUC

"To mitigate the risk of overfitting, the AUC value under 10-fold CV served as the performance evaluation metric."

- **Inside the training set:**
  - **Data were split into 10 folds**
  - **Models were trained/validated repeatedly**
- **AUC (Area Under ROC Curve) was used as the criterion to compare models and feature sets.**

This helps ensure the model generalises.

The authors split the data into training and test sets, performed feature selection exclusively on the training data to avoid leakage, and used 10-fold cross-validation with AUC as the evaluation metric, applying both embedded LASSO and random-forest-based recursive feature elimination to identify optimal predictor subsets.

After completing the thesis, for my paper, I would like to add

1) **Hyperparameter tuning**
2) **K-fold cross-validation**
3) **Add DL or ANN model**
4) **Shap, lime interpretation**
5) **More feature selection and extraction methods, and more advanced data preprocessing.**
6) **Logistic regression model**
7) **Doing CCA and group lasso—the feature selection thing**
8) **Studying more multivariate categorical variable selection methods.**
9) **Doing MCA and seeing if the feature extraction works**

Another important fact that I need to save and work on it later. 👍

The `birth_interval_months` variable is correctly structured in the DHS data. First births appropriately have `NaN` values because the preceding birth interval is undefined for them, which aligns with the DHS recode definition. The very large interval values (e.g., over 200 months) are biologically and socially plausible in Bangladesh and therefore should not be treated as errors or replaced. No impossible values were detected. For modelling, it is methodologically sound to create a separate indicator variable for first births and then handle the missing intervals accordingly, rather than imputing them without distinction.

Perfect. 👏
This is exactly what we wanted to see.

✅ Interpretation of Your Output

For:

```
df_ml[df_ml['birth_order'] == 1]['birth_interval_months'].describe()
```

You got:

count = 0

mean = NaN

This means:

✔ First births have **no value** for birth_interval
✔ They are stored as NaN
✔ DHS structure is preserved correctly

This is correct and clean.

🎯 Important Conclusion

Your dataset is structurally sound.

The long values (200+ months) we saw earlier:

✔ Are only for non-first births
✔ Are biologically plausible
✔ Are not data entry errors

So there are **no impossible values** in this variable.

🚨 Now the Real Modeling Question

You currently have:

- birth_interval = NaN for first births
- birth_interval = numeric for others

How are you handling these NaNs?

Because now this is a modeling design decision, not a cleaning issue.

Good — now we are talking about **model-level improvement**, not just feature selection.

You mentioned:

- Aggregating
- Bagging
- Bootstrapping

- Ensemble learning

Let's structure this clearly and technically.

🔹 1️⃣ Bootstrapping

Bootstrapping = sampling with replacement from training data.

This creates slightly different datasets.

Decision Trees are high variance.
Bootstrapping reduces variance.

Random Forest already does this internally.

So:

👉 If you are using Random Forest, you are already bootstrapping.

🔹 2️⃣ Bagging (Bootstrap Aggregating)

Bagging =

1. Train multiple models on bootstrapped samples
2. Average their predictions

Mathematically:

$$
\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^{(b)}(x)
$$

Decision Trees benefit greatly from bagging.

Random Forest = Bagging + random feature selection.

So if you already use RF, plain bagging adds little.

Unless:

- Your RF is shallow
- Or you use bagging on ExtraTrees

🔹 3️⃣ Aggregating (General Concept)

Aggregation means combining predictions.

There are different types:

- Mean (for probabilities)
- Majority vote
- Weighted vote
- Stacking

Aggregation reduces variance and increases stability.

🔹 4️⃣ Ensemble Learning (Broad Category)

Ensemble = combine multiple models.

There are 3 main types:

A) Bagging (Parallel, variance reduction)

Example:

- Random Forest
- ExtraTrees

Best for:

- High variance models (like Decision Tree)

B) Boosting (Sequential, bias reduction)

Example:

- XGBoost
- LightGBM
- AdaBoost

Best for:

- Improving AUC
- Handling complex patterns

C) Stacking (Meta-learning)

Train:

- RF
- XGB
- ExtraTrees

Then train a meta-model on their outputs.

This often gives best performance.

🔬 Now Let's Be Practical

You said:

- No deep learning
- Tree models only
- No leakage
- Clean workflow

Here are the realistic improvements available to you.

🔥 Option 1: Add ExtraTreesClassifier

ExtraTrees is like RF but:

- More randomness
- Lower variance
- Sometimes better recall

Very simple to test.

🔥 Option 2: Soft Voting Ensemble

Combine:

- Best RF
- Best XGB
- Best ExtraTrees

VotingClassifier(voting='soft')

Soft voting = average probabilities.

This often improves AUC by 1–3%.

🔥 Option 3: Balanced Random Forest

Instead of SMOTE:

Use:

from imblearn.ensemble import BalancedRandomForestClassifier

This bootstraps each tree with class-balanced samples.

Often improves recall for minority class.

Clean approach.

🔥 Option 4: Stacking (Advanced but Powerful)

Use:

- RF
- XGB
- ExtraTrees

Then:

Meta-model = Logistic Regression (only on model outputs)

This often increases AUC and F1.

But:

Requires careful CV to avoid leakage.

⚖️ Which One Fits Your Situation?

You have:

- Moderate imbalance (24%)
- 17 predictors
- 1743 training rows

You are not in high-dimensional setting.

So variance reduction matters.

🧠 My Ranked Recommendation

1️⃣ Tune RF properly
2️⃣ Add ExtraTrees
3️⃣ Try soft voting
4️⃣ Optimize threshold
5️⃣ Only then consider stacking

Not the other way around.

🚨 Important Warning

Ensemble methods improve:

- AUC
- Stability

But if recall is collapsing, the problem is usually:

- Threshold too high
- No class_weight
- Leaf size too small

Not lack of ensemble.

🎯 Key Insight

At your dataset size:

Hyperparameter tuning + threshold optimization

> usually gives bigger gains than complex ensemble tricks.

Now answer me this:

What is your current best test AUC and recall?

If AUC is already >0.75 and recall is low → threshold problem.
If AUC <0.70 → model capacity issue.
If Train AUC >> Test AUC → overfitting.

Tell me those numbers.

🧠 Scientifically Best Practice

You have two strong options:

Option 1 (Recommended for ML thesis)

Create an indicator:

```
df_ml['is_first_birth'] = (df_ml['birth_order'] == 1).astype(int)
```

Then:

Fill NaN in birth_interval with 0:

```
df_ml['birth_interval_months'] = df_ml['birth_interval_months'].fillna(0)
```

Why this works:

- 0 will only appear when is_first_birth = 1
- The model can distinguish:
    - First birth
    - Short interval
    - Long interval

Without the indicator, imputing distorts meaning.

Option 2

Keep NaN and use tree models that can handle missing (XGBoost can).

But:

Logistic regression cannot handle NaN.

📊 Why This Matters for Stunting

Birth spacing is strongly associated with:

- Maternal nutrient depletion
- Household resource allocation
- Child competition
- Infection exposure

And firstborn children are biologically different.

So your model must be able to distinguish them.

Excellent question.
Now we are thinking like a serious ML researcher.

You already know:

- **Union** → keep anything selected by at least one method
- **Intersection** → keep only variables selected by all methods

But those are just two extremes.

Let's systematically map all possible feature-combination strategies.

🧠 Suppose You Used 3 Feature Selection Methods:

Let's call them:

- FS1
- FS2
- FS3

Now here are your structured options:

1️⃣ Union (Liberal Strategy)

Keep:

$FS1 \cup FS2 \cup FS3$

Meaning: selected by at least one.

Pros

- High recall potential
- Low risk of dropping important predictors
- Good for tree models

Cons

- May include noisy variables
- Slight overfitting risk

2️⃣ Full Intersection (Conservative Strategy)

Keep:

FS1 ∩ FS2 ∩ FS3

Meaning: selected by all.

Pros

- High stability
- Low variance
- Often better generalization

Cons

- Risk of dropping useful nonlinear predictors
- May reduce recall

3️⃣ Majority Vote (Balanced Strategy) ⭐

Keep variables selected by **at least 2 out of 3 methods**.

This is often the best compromise.

Mathematically:

Selected if count ≥ 2

Why this is powerful:

- Removes weak single-method noise
- Keeps consensus predictors
- Very stable in small/medium datasets (like yours)

This is usually better than full intersection.

---

4 Weighted Voting Strategy

Assign weights to methods.

Example:

- Boruta = 2 points
- RF importance = 1 point
- Mutual info = 1 point

Keep features with score ≥ threshold.

This is advanced but clean.

---

5 Performance-Based Recursive Elimination

Start with union (17 features).

Then:

1. Fit tuned RF.
2. Rank by importance.
3. Drop bottom 2 features.
4. Re-train.
5. Compare AUC.
6. Stop when performance drops.

This is controlled backward elimination.

Very scientific.

---

6 Stability Selection

Repeat feature selection across CV folds.

Keep features that appear in ≥ 70% of folds.

This gives you robust predictors.

More computational but very publishable.

---

7 Embedded Selection via Regularized Model

Even though you're not using logistic regression now,
L1 (Lasso) logistic can act purely as selector.

Then use selected features in RF.

This is hybrid strategy.

🔎 What Is Best For Your Case?

You have:

- 1,743 training rows
- 17 features
- Tree-based models
- Moderate imbalance

Dimension is not high.

So:

I would NOT use full intersection.

Too aggressive.

I recommend:

Majority vote (≥ 2 methods).

It gives:

- Stability
- Better generalization
- Cleaner feature set
- Often improved recall stability

📊 Strategy Ranking For You

Best → Worst (for your dataset size):

1. Majority vote (≥2 methods)
2. Union (already done)
3. Recursive elimination from union
4. Weighted voting
5. Full intersection

🎯 What I Would Do Next

Create:

- Union set (17)
- Majority set (maybe 10–14 features)

Tune RF on both.

Compare:

- Test AUC
- Recall (class 1)
- CV variance

That's a clean experimental comparison.

⚠ Important Insight

With tree models, hyperparameter tuning often has more impact than small feature differences at this scale.

So don't expect 5–10% jumps from switching union to intersection.

Expect 1–3%.

The big gains come from:

- class_weight
- min_samples_leaf tuning
- threshold tuning

If you tell me the exact sizes of:

FS1
FS2
FS3

I can simulate which strategy likely gives best bias-variance tradeoff for you.

🔬 Final Verdict on This Variable

Issue	Status
Impossible values	❌ None
Biological plausibility	✔ Valid
Skewness	✔ Acceptable
Scaling needed	Already robustl scaled
Structural missing	✔ Correct

This answer is from outliers and stunting analysis.

And also, I will dive deeper into numerical variable.

🔹 Reason A: They have very small range

If:

- birth_order ranges 1–5
- children_ever_born ranges 1–6

Scaling barely changes anything.

In that case, leaving them unscaled is harmless.

🔹 Reason B: They are ordinal counts with meaningful step interpretation

Scaling removes interpretability.

Example:

- birth_order = 1 → first child
- birth_order = 2 → second child

After scaling, that meaning becomes abstract.

If your goal includes interpretability (especially logistic regression), keeping raw counts can be useful.

The chat in my project—model improvement strategy—holds important information about my fit to ml model step.